GlicoNutri

DOCUMENTAÇÃO TÉCNICA

Back-end

API REST, modelo de dados, regras de negócio e decisões de implementação do GlicoNutri

PREPARADO PARA

Mateus Rossini Marques Pêgo

PROJETO

GlicoNutri · TCC em parceria com a ADJ de Birigui

CAMADA

API em C# / ASP.NET sobre PostgreSQL

DOCUMENTO

Referência técnica do back-end

EMITIDO EM

10 de novembro de 2026

Sumário

1. Arquitetura — Camadas, fluxo de uma requisição e correspondência com o C4
2. Tecnologias e dependências — O que foi escolhido e por quê
3. Modelo de dados — As tabelas, herança, exclusão lógica e constraints
4. Segurança e autenticação — Credenciais, sessão, bloqueio e controle de acesso
5. Cálculos clínicos — As fórmulas implementadas, termo a termo
6. Regras de negócio — Onde cada uma das 35 é aplicada
7. Serviços — Responsabilidade de cada um
8. Superfície da API — Os 81 endpoints, agrupados por controlador
9. Importação da tabela TACO — O que o arquivo exige do leitor
10. Modelo de leitura do painel — Por que o dashboard não consulta as tabelas de escrita
11. Testes — Cobertura e verificação por mutação
12. Configuração e execução — Variáveis, segredos e como subir
13. Defeitos corrigidos — Erros que só apareceram com o sistema em execução
14. Dívida técnica — O que ficou pendente e o custo de cada item

1 Arquitetura

A API concentra toda a regra de negócio. O sistema web e o aplicativo mobile são clientes: nenhum deles acessa o banco diretamente, e nenhum reimplementa validação clínica. Essa escolha mantém a regra num lugar só e faz o aplicativo herdar, pronto, tudo que já foi validado pela web.

Camadas

CAMADA	RESPONSABILIDADE
Controllers	Recebem a requisição, validam o formato e delegam. Não contêm regra de negócio: no máximo decidem o código de resposta e verificam a permissão de acesso ao paciente. São 15, um por área.
Services	Onde vive a regra. Cada um responde por um caso de uso ou um agregado, e é onde as regras de negócio estão codificadas e comentadas com o número correspondente. São 24, somando cerca de 4.400 linhas.
Repositories e DbContext	Acesso a dados via Entity Framework Core. O DbContext concentra o mapeamento, os filtros de exclusão lógica e a conversão de nomes para o padrão do banco.
Models	As entidades do domínio, espelhando o diagrama de classes e o modelo entidade-relacionamento.

O caminho de uma requisição

ETAPA	O QUE ACONTECE
1. Autenticação	O middleware de JWT valida assinatura, emissor, audiência e validade. Um token inválido nunca chega ao controlador.
2. Senha provisória	Um middleware próprio recusa qualquer rota que não seja a troca de senha enquanto a credencial ainda for a provisória gerada pelo sistema.
3. Autorização por perfil	As políticas de Paciente, Nutricionista e Administrador são verificadas pelo ASP.NET. O administrador recebe também a função de nutricionista no token, o que resolve a herança de permissões sem regra condicional espalhada.
4. Permissão sobre o paciente	Para rotas clínicas, um serviço dedicado decide se o usuário pode acessar aquele paciente: o próprio paciente, o nutricionista vinculado ou o administrador.
5. Regra de negócio	O serviço aplica as validações e persiste. Regras que dependem do estado do banco — como a unicidade de plano ativo — são reforçadas por constraint.
6. Modelo de leitura	Serviços que escrevem dado clínico atualizam a tabela de resumo usada pelo painel, mantendo o dashboard isolado das tabelas transacionais.

Correspondência com o diagrama de componentes

Os componentes desenhados no C4 e o que existe no código:

NO DIAGRAMA	NO CÓDIGO
`JwtMiddleware`	Autenticação JWT do ASP.NET, configurada em Program.cs
`RbacMiddleware`	Políticas de autorização por perfil, mais o middleware de senha provisória
`AuthController e AuthService`	Implementados com o mesmo nome

NO DIAGRAMA	NO CÓDIGO
`PacienteService e NutricionistaService`	Implementados com o mesmo nome
`GlicemiaService, AntropometriaService, EmocaoService`	Os dois primeiros com o mesmo nome; o terceiro chama-se RegistroEmocionalService
`CalculadoraNutricionalService`	Implementado como CalculadoraNutricional, classe estática sem estado — os cálculos são funções puras
`ImportadorTACOService`	Implementado como ImportadorTacoService, com a leitura do arquivo isolada em LeitorCsvTaco
`PlanoAlimentarService, AlertaService, DashboardService, RelatorioService`	Implementados com o mesmo nome
`ConteudoEducativoService`	Implementado como RepositorioEducativoService, cobrindo conteúdos e receitas

2 Tecnologias e dependências

As escolhas seguem o que o projeto de pesquisa definiu, com uma substituição no banco.

CAMADA	ESCOLHA	MOTIVO
Linguagem e framework	`.NET 10 · C#`	Definido no projeto de pesquisa. A versão 10 é a mais recente com suporte de longo prazo.
Acesso a dados	`Entity Framework Core 10 · Npgsql 10`	ORM oficial da plataforma, com o provedor PostgreSQL. As migrations versionam o schema.
Banco de dados	`PostgreSQL 17 no Supabase`	O projeto de pesquisa previa PostgreSQL. O Supabase entra como serviço gerenciado desse mesmo banco: a API continua acessando por conexão direta e Entity Framework, sem usar a API própria do Supabase, o que mantém o diagrama de containers verdadeiro.
Autenticação	`JWT Bearer · BCrypt.Net-Next`	Token próprio emitido pela API. O bcrypt atende à exigência de hash seguro.
Leitura de CSV	`CsvHelper`	Biblioteca citada no relatório de atividades da semana 20 como caminho para importar a tabela nutricional.
Geração de PDF	`QuestPDF`	Licença Community, gratuita para este uso. Gera os relatórios clínicos previstos no diagrama de classes.
Envio de e-mail	`MailKit`	Biblioteca recomendada para SMTP em .NET.
Login federado	`Google.Apis.Auth`	Valida o token de identidade emitido pelo Google, conforme a regra de negócio 01.
Containerização	`Docker · Docker Compose`	O Postgres de desenvolvimento sobe em container, o que dispensa instalar banco na máquina de quem for trabalhar no projeto. A API também é publicada como imagem: é a containerização que define quais hospedagens gratuitas conseguem receber o sistema.
Testes	`xUnit · Microsoft.AspNetCore.Mvc.Testing`	Testes unitários e de integração, estes últimos subindo a aplicação inteira contra um banco descartável.

3 Modelo de dados

O modelo segue o diagrama entidade-relacionamento versão 2.0. Os nomes de tabela e coluna do banco reproduzem exatamente os do documento. São 30 tabelas de domínio, mais a tabela de controle das migrations, mantida pelo Entity Framework.

Grupos de tabelas

Identidade e autenticação

`usuarios, nutricionistas, pacientes, administradores, nutricionista_paciente`

Hierarquia com herança Table-Per-Hierarchy: uma única tabela usuarios guarda paciente, nutricionista e administrador, e a coluna de perfil diz o que cada linha é. As tabelas por subtipo do modelo entidade-relacionamento versão 2.0 deixaram de existir.

Plano alimentar

`planos_alimentares, distribuicao_macronutrientes, itens_plano_alimentar, necessidades_energeticas`

O plano referencia paciente e nutricionista, tem exatamente uma distribuição de macronutrientes e vários itens de refeição.

Banco de alimentos

`alimentos, receitas, ingredientes_receita`

Os ingredientes de uma receita apontam para alimentos cadastrados, o que permite calcular os valores nutricionais a partir da base.

Registros clínicos

`registros_glicemia, registros_antropometricos, registros_emocionais`

Séries temporais por paciente, com índice composto de paciente e data para as consultas por período.

Alertas

`alertas, historico_alertas`

A configuração fica em alertas; cada disparo e seu desfecho ficam no histórico.

Conteúdo educativo

`conteudos_educativos`

Artigos, dicas, receitas e vídeos publicados pelos profissionais.

Modelo de leitura

`resumo_clinico_paciente (view)`

Uma linha por paciente, alimentada pelos serviços de escrita e consumida apenas pelo painel.

Tabelas de referência

`perfis_usuario, sexos_biologicos, tipos_diabetes, contextos_glicemia, estados_emocionais, formulas_energeticas, niveis_atividade, tipos_conteudo, tipos_alerta, status_envio, fontes_alimento`

Substituem as enumerações estáticas, conforme a mudança C1 do diagrama de classes versão 5.0. Todas seguem o mesmo formato: identificador, código, descrição e situação.

Decisões de mapeamento

DECISÃO	DETALHE
Herança Table-Per-Hierarchy	O discriminador é a própria coluna de perfil, que já era chave estrangeira obrigatória: uma segunda coluna com o tipo diria o mesmo e poderia divergir dela. Em troca, o que era obrigatório apenas em um subtipo não pode mais ser NOT NULL, porque a coluna fica nula nas linhas dos outros perfis — a obrigatoriedade virou verificação condicionada ao perfil, declarada no banco.
Exclusão lógica com filtro global	Doze entidades têm o campo de situação e um filtro global que as esconde quando inativas. Dependentes sem campo próprio — histórico de alertas, distribuição de macronutrientes, ingredientes e o resumo clínico — herdam o filtro do principal, senão sobreviveriam à exclusão de quem os governa.
Alimentos fora do filtro global	A regra de negócio 12 é específica: alimentos inativados somem das buscas, mas seus dados são preservados para os planos que os referenciam. Um filtro global os esconderia também quando alcançados por um item de plano, quebrando justamente o histórico que a regra manda preservar. A exclusão é aplicada nas consultas de busca.
Conversão de nomes para o padrão do banco	Um conversor transforma os nomes das classes em snake_case, reproduzindo o documento. Os dígitos exigiram regra própria: separador antes de um número precedido de letra, e nunca entre o número e a letra seguinte. Sem isso saíam calorias_por100g e media_glicemia7_dias, divergindo do modelo.
Regras críticas no próprio banco	A unicidade de plano ativo por paciente é um índice único parcial, e as faixas de glicemia e de intensidade emocional são restrições de verificação. A regra vale mesmo que a aplicação erre.

As 31 tabelas

TABELA	COLUNAS	CHAVE PRIMÁRIA	FKS	CHECKS
`__EFMigrationsHistory`	2		0	
`alertas`	8	`id`	2	
`alimentos`	12	`id`	2	
`conteudos_educativos`	8	`id`	2	
`contextos_glicemia`	4	`id`	0	
`distribuicao_macronutrientes`	9	`id`	1	
`estados_emocionais`	4	`id`	0	
`favoritos_conteudo`	3	`conteudo_id, paciente_id`	2	
`favoritos_receita`	3	`paciente_id, receita_id`	2	
`fontes_alimento`	4	`id`	0	
`fontes_indice_glicemico`	4	`id`	0	
`formulas_energeticas`	4	`id`	0	
`historico_alertas`	7	`id`	2	
`ingredientes_receita`	5	`id`	2	
`itens_plano_alimentar`	9	`id`	2	
`necessidades_energeticas`	8	`id`	3	
`niveis_atividade`	5	`id`	0	

TABELA	COLUNAS	CHAVE PRIMÁRIA	FKS	CHECKS
nutricionista_paciente	4	nutricionista_id, paciente_id	2	
perfis_usuario	4	id	0	
planos_alimentares	9	id	2	
receitas	9	id	1	
registros_antropometricos	13	id	1	
registros_emocionais	7	id	2	1
registros_glicemia	8	id	2	1
resumo_clinico_paciente	13		0	
sexos_biologicos	4	id	0	
status_envio	4	id	0	
tipos_alerta	4	id	0	
tipos_conteudo	4	id	0	
tipos_diabetes	4	id	0	
usuarios	25	id	3	

Detalhe de tabelas centrais

usuarios

COLUNA	TIPO	NOTAS
id	bigint	obrigatório
nome	varchar(150)	obrigatório
email	varchar(150)	obrigatório
senha_hash	varchar(255)	obrigatório
perfil_id	bigint	→ perfis_usuario.id · obrigatório
ativo	bool	obrigatório
data_cadastro	timestamptz	obrigatório · padrão now()
ultimo_acesso	timestamptz	
tentativas_invalidas	int	obrigatório
nivel_bloqueio	int	obrigatório
bloqueado_ate	timestamptz	
senha_provisoria	bool	obrigatório
cpf	text	
crn	varchar(20)	
data_nascimento	date	
especialidade	varchar(100)	
glicemia_max_alvo	float	
glicemia_min_alvo	float	

COLUNA	TIPO	NOTAS
medicacao_em_uso	`text`	
nivel_acesso	`int`	
observacoes_clinicas	`text`	
sexo_id	`bigint`	→ sexos_biologicos.id
telefone	`text`	
tipo_diabetes_id	`bigint`	→ tipos_diabetes.id
cpf_hash	`varchar(64)`	

planos_alimentares

COLUNA	TIPO	NOTAS
id	`bigint`	obrigatório
paciente_id	`bigint`	→ usuarios.id · obrigatório
nutricionista_id	`bigint`	→ usuarios.id · obrigatório
data_inicio	`date`	obrigatório
data_fim	`date`	
objetivo	`varchar(255)`	
observacoes	`text`	
total_calorias_prescritas	`float`	
ativo	`bool`	obrigatório

registros_glicemia

COLUNA	TIPO	NOTAS
id	`bigint`	obrigatório
paciente_id	`bigint`	→ usuarios.id · obrigatório
valor	`float`	obrigatório
contexto_id	`bigint`	→ contextos_glicemia.id · obrigatório
data_hora	`timestamptz`	obrigatório
observacao	`text`	
fora_do_alvo	`bool`	obrigatório
ativo	`bool`	obrigatório

resumo_clinico_paciente

COLUNA	TIPO	NOTAS
paciente_id	`bigint`	
ultima_glicemia_valor	`float`	
ultima_glicemia_contexto_id	`bigint`	
ultima_glicemia_data	`timestamptz`	
media_glicemia_7dias	`float`	

COLUNA	TIPO	NOTAS
percentual_no_alvo_7dias	`float`	
ultimo_imc	`float`	
ultima_classificacao_imc	`varchar(50)`	
ultima_data_antropometria	`timestamptz`	
plano_ativo	`bool`	
alertas_pendentes_count	`int`	
dias_sem_registro_glicemia	`int`	
data_atualizacao	`timestamptz`	

favoritos_conteudo

COLUNA	TIPO	NOTAS
paciente_id	`bigint`	→ usuarios.id · obrigatório
conteudo_id	`bigint`	→ conteudos_educativos.id · obrigatório
data_favoritado	`timestamptz`	obrigatório · padrão now()

Restrições de verificação

Regras clínicas gravadas como constraint, válidas mesmo que a aplicação erre:

TABELA	NOME	CONDIÇÃO
`registros_emocionais`	`ck_registros_emocionais_intensidade`	`CHECK (((intensidade >= 1) AND (intensidade <= 5)))`
`registros_glicemia`	`ck_registros_glicemia_valor`	`CHECK (((valor >= (0)::double precision) AND (valor <= (600)::double precision)))`

Índices parciais

TABELA	DEFINIÇÃO
`ix_planos_alimentares_paciente_ativo_unico`	`CREATE UNIQUE INDEX ix_planos_alimentares_paciente_ativo_unico ON public.planos_alimentares USING btree (paciente_id) WHERE (ativo = true)`

Migrations aplicadas

- `20260906203129_InicialDerV2`
- `20260906204530_RenomearColunasHorarioAlerta`
- `20260907180533_ExtensaoUnaccent`
- `20260907182023_CorrigeNomesDeColunasComDigito`
- `20260912054001_HerancaTph`
- `20260912054440_ResumoClinicoComoView`
- `20260912055530_IndiceGlicemicoComOrigemEPreDiabetes`
- `20260912055952_FavoritosEMidiaDoConteudo`
- `20261109185431_ProtecaoContraApiPublicaDoSupabase`
- `20261110221409_IndiceGlicemicoTabelaInternacional2021`
- `20261110222912_CriptografiaPorColunaRnf01`

4 Segurança e autenticação

A API é a única fronteira de segurança. Nenhum cliente acessa o banco, e a verificação de permissão acontece no servidor mesmo quando a interface já a espelha.

ASPECTO	IMPLEMENTAÇÃO
Armazenamento de credenciais	Senhas apenas em hash bcrypt, com fator de custo 12 — cerca de 250 milissegundos por verificação no hardware atual, o que torna a força bruta cara. O salt por hash impede tabelas pré-computadas. Um hash corrompido é tratado como credencial inválida em vez de derrubar a requisição.
Sessão	Token JWT assinado com HMAC-SHA256, válido por 24 horas, contendo identificador, e-mail, perfil e o indicador de senha provisória. É renovado enquanto o sistema está em uso, para a sessão não cair no meio de um atendimento.
Bloqueio progressivo	Cinco tentativas inválidas consecutivas bloqueiam a conta. A espera cresce a cada bloqueio: cinco minutos, trinta minutos e, do terceiro em diante, vinte e quatro horas. O nível é preservado após o desbloqueio, para que o próximo seja mais longo. O acerto da senha zera o ciclo. O tempo restante volta na resposta, para a tela informar.
Recuperação de senha	O link vale trinta minutos e é de uso único sem tabela de tokens: ele carrega uma impressão digital do hash atual da senha, que deixa de conferir assim que a senha muda. Continua disponível com a conta bloqueada, e concluí-lo libera o acesso.
Não revelação de contas	Login com e-mail inexistente, com conta desativada ou com senha errada devolvem a mesma mensagem. A recuperação responde igual exista ou não a conta.
Controle de acesso por perfil	Três políticas, verificadas pelo ASP.NET. O administrador recebe também a função de nutricionista no token: um endpoint marcado para nutricionista passa a aceitá-lo sem condicional espalhada pelo código.
Permissão sobre o paciente	Rotas clínicas passam por um serviço que decide caso a caso: o próprio paciente, o nutricionista com vínculo ativo, ou o administrador. É o que implementa a proteção de dados de saúde exigida pela regra 33.
Login federado	O cliente obtém o token de identidade no Google e o envia à API, que valida assinatura e audiência. Não cria conta: e-mail sem usuário ativo recebe mensagem informativa e acesso negado. O bloqueio por tentativas vale igualmente por essa via, para que trocar de forma de entrada não seja atalho.
Criptografia por coluna	CPF, telefone, observações clínicas e medicação em uso são cifrados pela própria API antes de gravar, com AES-256-GCM e nonce aleatório — além da criptografia de disco do Supabase. Como a cifra muda a cada gravação, o CPF ganha uma segunda coluna com hash HMAC-SHA256, determinístico e irreversível, que carrega o índice único e a busca por duplicidade. Nome, glicemias e plano ficam sem cifra por coluna: cifrá-los quebraria busca, ordenação e as agregações do painel. Requisito não funcional 01.
Cabeçalhos de resposta	Toda resposta sai com X-Content-Type-Options, X-Frame-Options e Referrer-Policy, contra interpretação indevida de conteúdo, embutimento em outra página e vazamento de endereço.
Segredos fora do repositório	Connection string, chave de assinatura, credenciais de e-mail e identificador do Google ficam no armazenamento de segredos do .NET. O repositório traz apenas o arquivo de exemplo.

5 Cálculos clínicos

Os cálculos que sustentam a prescrição, com os coeficientes exatos. Todos são funções puras, sem estado, e é sobre eles que a maior parte dos testes incide.

Índice de massa corporal

```
IMC = peso (kg) ÷ altura (m)²
```

A altura é persistida em centímetros, como o caso de uso 008 a coleta, e convertida no cálculo. O valor aceito vai de 50 a 250 centímetros: a faixa recusa altura digitada em metros, um erro que passaria despercebido e falsearia todo o resultado.

Classificação do índice

```
abaixo de 18,5 · de 18,5 a 24,9 · de 25 a 29,9 · 30 ou mais
```

Abaixo do peso, peso normal, sobrepeso e obesidade, conforme o fluxo A2 do caso de uso 008. A Organização Mundial da Saúde ainda subdivide a obesidade em graus; estender as faixas é uma linha, se a equipe de Nutrição preferir.

Relação cintura-quadril

```
RCQ = circunferência da cintura ÷ circunferência do quadril
```

Calculada apenas quando as duas medidas existem. Faltando uma, o campo fica vazio em vez de receber um valor derivado de dado ausente.

Harris-Benedict — masculino

```
TMB = 88,362 + (13,397 × peso) + (4,799 × altura) − (5,677 × idade)
```

Revisão de Roza e Shizgal, de 1984.

Harris-Benedict — feminino

```
TMB = 447,593 + (9,247 × peso) + (3,098 × altura) − (4,330 × idade)
```

A equação original, de 1919, usa coeficientes diferentes — 66,473, 13,7516, 5,0033 e 6,755 para homens; 655,0955, 9,5634, 1,8496 e 4,6756 para mulheres — e ainda é ensinada em parte dos cursos. Qual das duas vale é decisão da equipe de Nutrição.

Mifflin-St Jeor

```
TMB = (10 × peso) + (6,25 × altura) − (5 × idade) + 5 para homens, ou − 161 para mulheres
```

Segunda fórmula prevista pela regra 14. A diferença entre os sexos é apenas o termo constante.

Valor energético total

```
VET = TMB × fator de atividade
```

Os fatores acompanham a tabela de referência de níveis de atividade: 1,2 para sedentário; 1,375 levemente ativo; 1,55 moderadamente ativo; 1,725 muito ativo; 1,9 extremamente ativo. Guardar o fator como coluna da tabela de referência foi consequência de converter as enumerações em entidades — ele é dado de domínio, e não caberia num tipo do banco.

Distribuição de macronutrientes

```
gramas = VET × percentual ÷ fator calórico
```

Fatores de Atwater: 4 quilocalorias por grama de carboidrato e de proteína, 9 por grama de lipídio. A distribuição padrão é 55, 20 e 25 por cento, que fica dentro das faixas do caso de uso 007 e soma exatamente cem.

Contribuição de um item do plano

```
valor = valor por 100 g × quantidade ÷ 100
```

A quantidade é registrada em gramas. Valor ausente na base não vira zero: a soma o ignora e o sistema sinaliza que o total está subestimado.

Classificação glicêmica

```
abaixo de 70 · de 70 a 180 · acima de 180
```

Hipoglicemia, normal e hiperglicemia, por limiar clínico absoluto, conforme a regra 02 do caso de uso 004. É informação distinta da faixa alvo individual, e as duas convivem.

Marcação de fora do alvo

```
fora = valor < mínimo do paciente, ou valor > máximo do paciente
```

Usa a faixa individual definida pelo nutricionista. Sem personalização, aplica-se o padrão clínico de 70 a 180 e o sistema sinaliza a pendência. Um valor de 168 num paciente com alvo estreito está fora do alvo sem ser hiperglicemia.

Idade em anos completos

```
diferença de anos, subtraindo um se o aniversário ainda não ocorreu
```

Para quem nasceu em 29 de fevereiro, o sistema segue o comportamento padrão da plataforma, que ajusta a data para 28 de fevereiro em anos não bissextos. Não há convenção única; a escolhida está fixada por teste, e a diferença é de um dia a cada quatro anos.

6 Regras de negócio

Onde cada regra é aplicada. O código cita o número da regra no ponto exato em que ela é verificada, o que torna a rastreabilidade verificável por busca no repositório.

RN01	AuthService, DashboardService, RegistroEmocionalService
RN02	AuthService, CalculadoraNutricional
RN03	AlertaService, AuthService, DashboardService, NecessidadeEnergeticaService, NutricionistaService, PacienteService, PlanoAlimentarService, SenhaService
RN04	GlicemiaService, NecessidadeEnergeticaService, PlanoAlimentarService, RegistroEmocionalService, TokenService
RN05	AntropometriaService, AuthService, NutricionistaService, PacienteService, UsuarioRepository
RN06	NutricionistaService, PacienteService, UsuarioRepository
RN07	NutricionistaService, PacienteService, PlanoAlimentarService
RN08	PacienteService
RN09	NutricionistaService
RN10	ImportadorTacoService, LeitorCsvTaco
RN11	AlimentoService, ImportadorTacoService, LeitorCsvTaco
RN12	AlimentoService, RepositorioEducativoService
RN13	PlanoAlimentarService
RN14	CalculadoraNutricional, NecessidadeEnergeticaService
RN15	CalculadoraNutricional, PlanoAlimentarService
RN16	PlanoAlimentarService
RN17	PlanoAlimentarService
RN18	GlicemiaService
RN19	CalculadoraNutricional, GlicemiaService, PacienteService
RN20	CalculadoraNutricional, DashboardService, GlicemiaService, PacienteService, RelatorioService
RN21	AntropometriaService, CalculadoraNutricional
RN22	RegistroEmocionalService
RN23	AlertaService
RN24	PacienteService
RN26	AlertaService
RN27	AlertaService
RN28	RepositorioEducativoService
RN29	FavoritoService, RepositorioEducativoService
RN30	RepositorioEducativoService
RN32	AuthService, SenhaService
RN33	AcessoPacienteService, AlertaService, AntropometriaService, GlicemiaService, PacienteService, PlanoAlimentarService, RegistroEmocionalService
RN35	AlertaService

As regras 31 e 34 não aparecem porque não são de código: a primeira depende da publicação com HTTPS, e a segunda é um compromisso operacional de disponibilidade.

7 Serviços

Os 25 serviços da camada de negócio, ordenados por quantidade de operações. A coluna final traz o tamanho em linhas.

SERVIÇO	RESPONSABILIDADE E OPERAÇÕES	LINHAS
`CalculadoraNutricional`	Cálculos clínicos do sistema — o CalculadoraNutricional do Diagrama de Classes V5.0 e o CalculadoraNutricionalService do C4. ATENÇÃO CLÍNICA: os coeficientes daqui definem a prescrição nutricional e precisam de validação pela equi `CalcularImc, ClassificarImc, CalcularRcq, HarrisBenedict, MifflinStJeor, CalcularTmb, CalcularVet, CalcularIdade, ForaDoAlvo e mais 6` RN02 RN14 RN15 RN19 RN20 RN21	187
`RepositorioEducativoService`	RF09 — Repositório Educativo. Publicação restrita a Nutricionista e Administrador (RN28); despublicação exclusiva do Administrador, sempre por inativação lógica (RN29). Receita carrega ingredientes vinculados a alimentos cadastrad `CriarConteudoAsync, ListarConteudosAsync, ObterConteudoAsync, AtualizarConteudoAsync, DespublicarConteudoAsync, RepublicarConteudoAsync, CriarReceitaAsync, ListarReceitasAsync, ObterReceitaAsync e mais 3` RN12 RN28 RN29 RN30	351
`AlertaService`	UC010 — Configurar Alertas e Lembretes. A configuração é exclusiva do Nutricionista ou do Administrador (RN23); o paciente apenas desativa. `CriarAsync, ListarAsync, AtualizarAsync, DesativarAsync, ReativarAsync, RegistrarDisparoAsync, HistoricoAsync, VerificarAlertasExpiradosAsync` RN03 RN23 RN26 RN27 RN33 RN35	324
`PlanoAlimentarService`	UC007 — Elaborar Plano Alimentar. Encadeia as regras que sustentam a prescrição: VET obrigatório (RN13), macros somando 100% (RN15), plano ativo único (RN16) e histórico preservado (RN17). `CriarAsync, ObterAtivoAsync, ObterAsync, HistoricoAsync, SugerirDistribuicaoAsync, DuplicarAsync, ReativarAsync, InativarAsync` RN03 RN04 RN07 RN13 RN15 RN16	380
`PacienteService`	UC002 — Cadastrar Paciente. Ação restrita a Nutricionista e Administrador (RN08): o paciente nunca cria a própria conta. `CriarAsync, ListarAsync, BuscarAsync, DefinirMetasGlicemicasAsync, AtualizarAsync, DesativarAsync, ReativarAsync` RN03 RN05 RN06 RN07 RN08 RN19	291
`AlimentoService`	UC009 — Cadastrar Alimento. Cadastro manual (RF01.1), busca e inativação lógica (RN12). `CriarAsync, BuscarAsync, ObterAsync, GruposAsync, AtualizarAsync, InativarAsync` RN11 RN12	148
`AuthService`	UC001 — Login e Autenticação. Concentra a validação de credenciais (RN32), o bloqueio progressivo (RN02) e a troca obrigatória da senha provisória (RN03). `LoginAsync, LoginGoogleAsync, RenovarAsync, RecuperarSenhaAsync, RedefinirSenhaAsync, AlterarSenhaAsync` RN01 RN02 RN03 RN05 RN32	297
`NutricionistaService`	UC003 — Cadastrar Nutricionista. Ação exclusiva do Administrador (RN09). `CriarAsync, ListarAsync, BuscarAsync, AtualizarAsync, DesativarAsync, ReativarAsync` RN03 RN05 RN06 RN07 RN09	177
`FavoritoService`	RF09.3 — favoritos do paciente no repositório educativo. Marcar duas vezes o mesmo material não é erro: a chave é o par paciente + material, e a segunda marcação não tem efeito. Isso deixa a tela livre para tratar o botão como um `ListarAsync, MarcarConteudoAsync, DesmarcarConteudoAsync, MarcarReceitaAsync, DesmarcarReceitaAsync` RN29	130

SERVIÇO	RESPONSABILIDADE E OPERAÇÕES	LINHAS
`GlicemiaService`	UC004 — Registrar Glicemia e UC005 — Visualizar Histórico Glicêmico. Classifica cada registro contra a faixa alvo do paciente (RN19/RN20) e mantém o read model do dashboard atualizado. `RegistrarAsync, HistoricoAsync, SerieAsync, RemoverAsync, ReclassificarAsync` RN04 RN18 RN19 RN20 RN33	214
`LeitorCsvTaco`	Leitura do CSV da TACO/IBGE. Isolado do serviço de importação porque é aqui que moram as particularidades do arquivo — e são muitas. `DecodificarTexto, DetectarDelimitador, TentarNumero, MapearColunas, SemAcento` RN10 RN11	143
`AntropometriaService`	UC008 — Registrar Dados Antropométricos. Calcula e classifica o IMC (RN21), deriva a RCQ e mantém o read model do dashboard em dia. `RegistrarAsync, HistoricoAsync, RemoverAsync, SerieAsync` RN05 RN21 RN33	152
`RegistroEmocionalService`	UC012 — Registrar Emoção, e a análise de correlação do RF06.2. `RegistrarAsync, DiarioAsync, CorrelacaoAsync, RemoverAsync` RN01 RN04 RN22 RN33	184
`SenhaService`	RN32 — senhas só existem em hash bcrypt. Texto puro, MD5 e SHA-1 são vedados. `Hash, Conferir, GerarProvisoria, ImpressaoDigital` RN03 RN32	62
`UsuarioRepository`	Acesso a usuários, perfis e ao contador de tentativas de login (C4, módulo de Autenticação). `BuscarPorEmailAsync, BuscarPorIdAsync, EmailEmUsoAsync, SalvarAsync` RN05 RN06	41
`EmailService`	Implementação de desenvolvimento: registra a mensagem no log em vez de enviar. O envio real (RF10.2 e RF10.3) entra na Fase 1, trocando só esta classe. `EnviarRecuperacaoSenhaAsync, EnviarBoasVindasAsync, EnviarContaBloqueadaAsync`	41
`NecessidadeEnergeticaService`	UC006 — Calcular Necessidade Energética. Só fórmulas validadas (RN14), com a fórmula usada registrada em cada cálculo para rastreabilidade clínica. `SimularAsync, CalcularESalvarAsync, HistoricoAsync` RN03 RN04 RN14	173
`RelatorioService`	RelatorioService do Diagrama de Classes V5.0 e do C4. Ao contrário do DashboardService, este lê das tabelas de escrita — o C4 é explícito: consolida o histórico clínico completo, e isso só para relatórios exportáveis. `GerarRelatorioGlicemicoAsync, GerarRelatorioAntropometricoAsync, GerarRelatorioCompletoAsync` RN20	440
`TokenService`	`GerarTokenRecuperacao, ValidarTokenRecuperacao` RN04	115
`ValidadorCpf`	Validação do dígito verificador do CPF, exigida como fluxo <<include>> pelo UC002 (A1) e pelo UC003 (A1). `EhValido, Normalizar`	53
`AcessoPacienteService`	RN33 — dados clínicos são acessíveis apenas pelo próprio paciente, pelo Nutricionista responsável vinculado e pelo Administrador. Concentrar a regra aqui também concilia os documentos: o UC008 coloca o Nutricionista como ator do r `PodeAcessarAsync` RN33	43

SERVIÇO	RESPONSABILIDADE E OPERAÇÕES	LINHAS
`DashboardService`	UC011 — Dashboard do Nutricionista. Todos os indicadores clínicos vêm de resumo_clinico_paciente, o Read Model, como determinam a nota do DER V2.0 e o C4: nenhuma agregação é calculada sobre as tabelas de escrita. As únicas leitur `ObterAsync` RN01 RN03 RN20	188
`ImportadorTacoService`	RF01.2 / RN10 — importação em lote da tabela TACO/IBGE. Valida linha a linha antes de persistir e devolve relatório de erros sem interromper as linhas válidas. `ImportarAsync` RN10 RN11	216

8 Superfície da API

Os 87 endpoints, agrupados por controlador. A coluna de perfil indica a política exigida; onde diz autenticado, a permissão é decidida por paciente.

AlimentoController

UC009 — Cadastrar Alimento. Nutricionista ou Administrador; o Administrador já é contemplado pela política de Nutricionista por herança (RN04).

	ROTA	DESCRIÇÃO	PERFIL
GET	`/api/alimentos`	RN12 — a busca não devolve alimentos inativados.	Nutricionista
GET	`/api/alimentos/grupos`		Nutricionista
GET	`/api/alimentos/{id}`		Nutricionista
POST	`/api/alimentos`	RF01.1 — cadastro manual. `CriarAlimentoRequest`	Nutricionista
PUT	`/api/alimentos/{id}`	`CriarAlimentoRequest`	Nutricionista
DELETE	`/api/alimentos/{id}`	RN12 — inativação lógica; o alimento continua resolvível pelos planos.	Nutricionista
POST	`/api/alimentos/importar`	RF01.2 / RN10 — importação em lote via CSV no padrão TACO/IBGE. Devolve o relatório de erros junto com a contagem do que entrou. `arquivo`	Nutricionista

AuthController

UC001 — Login e Autenticação. Rotas públicas, exceto a troca de senha, que exige o usuário já autenticado.

	ROTA	DESCRIÇÃO	PERFIL
POST	`/api/auth/login`	Autentica o usuário e devolve o token de acesso válido por 24 horas. `LoginRequest`	Público
POST	`/api/auth/login-google`	RN01 — login federado com conta Google. O cliente envia o ID token obtido no Google Identity Services; o acesso só é concedido se o e-mail corresponde `LoginGoogleRequest`	Público
POST	`/api/auth/renovar`	RN03 do UC001 — renova o token de quem está em uso ativo, antes que a validade de 24 horas expire e derrube a sessão no meio do atendimento.	—
POST	`/api/auth/recuperar-senha`	Dispara o e-mail de redefinição de senha. Responde 202 mesmo quando o e-mail não existe, para não revelar quais contas estão cadastradas. `RecuperarSenhaRequest`	Público
POST	`/api/auth/redefinir-senha`	Redefine a senha a partir do link recebido por e-mail. `RedefinirSenhaRequest`	Público
POST	`/api/auth/alterar-senha`	Troca a senha do usuário autenticado. Única rota liberada enquanto a credencial ainda for a provisória (RN03). `AlterarSenhaRequest`	—
GET	`/api/auth/eu`	Dados do usuário autenticado, usados pelo front para montar o menu por perfil.	—

ConteudoEducativoController

RF09.2 — Conteúdo educativo. RN28 — criar e editar são de Nutricionista ou Administrador; o paciente tem permissão exclusiva de leitura. RN29 — despublicar é só do Administrador.

	ROTA	DESCRIÇÃO	PERFIL
GET	/api/conteudos	RF09.3 — leitura liberada a todos os perfis autenticados.	Autenticado
GET	/api/conteudos/{id}		Autenticado
POST	/api/conteudos	`CriarConteudoRequest`	Nutricionista
PUT	/api/conteudos/{id}	`CriarConteudoRequest`	Nutricionista
DELETE	/api/conteudos/{id}	RN29 — despublicação é competência exclusiva do Administrador.	Administrador
POST	/api/conteudos/{id}/republicar		Administrador

DashboardController

UC011 — Dashboard do Nutricionista. RF08.3.

	ROTA	DESCRIÇÃO	PERFIL
GET	/api/dashboard	RN01 do UC011 — o Nutricionista vê apenas os pacientes vinculados a ele. O Administrador, que herda o perfil pela RN04, enxerga todos e pode filtrar p	Nutricionista

NutricionistaController

UC003 — Cadastrar Nutricionista. Todo o controller exige perfil de Administrador: a RN09 veda que um Nutricionista crie contas de outros.

	ROTA	DESCRIÇÃO	PERFIL
POST	/api/nutricionistas	`CriarNutricionistaRequest`	Administrador
GET	/api/nutricionistas		Administrador
GET	/api/nutricionistas/{id}		Administrador
PUT	/api/nutricionistas/{id}	RF10.5 — edição dos dados do nutricionista. `AtualizarNutricionistaRequest`	Administrador
DELETE	/api/nutricionistas/{id}	RF10.5 — desativa sem excluir o histórico (RN05).	Administrador
POST	/api/nutricionistas/{id}/reativar		Administrador

PacienteController

UC002 — Cadastrar Paciente. Perfil de Nutricionista ou Administrador (RN08). Como o Administrador também carrega a role de Nutricionista no token (RN04), a política de Nutricionista já o contempla.

	ROTA	DESCRIÇÃO	PERFIL
POST	/api/pacientes	`CriarPacienteRequest`	Nutricionista
GET	/api/pacientes	RN33 — o Nutricionista lista somente os próprios pacientes. O Administrador enxerga todos, e pode filtrar por profissional responsável.	Nutricionista
GET	/api/pacientes/{id}		Nutricionista
PUT	/api/pacientes/{id}	RF10.5 — edição dos dados cadastrais do paciente. `AtualizarPacienteRequest`	Nutricionista
PUT	/api/pacientes/{id}/metas-glicemicas	RN20 — define a faixa glicêmica alvo individual do paciente. `DefinirMetasGlicemicasRequest`	Nutricionista
DELETE	/api/pacientes/{id}		Nutricionista

	ROTA	DESCRIÇÃO	PERFIL
POST	/api/pacientes/{id}/reativar		Nutricionista

AlertaController

UC010 — Configurar Alertas e Lembretes. RN23 — criar, editar e excluir são exclusivos do Nutricionista ou do Administrador; ao paciente cabe apenas desativar pelo aplicativo.

	ROTA	DESCRIÇÃO	PERFIL
GET	/api/pacientes/{pacienteId}/alertas		Autenticado
POST	/api/pacientes/{pacienteId}/alertas	`CriarAlertaRequest`	Nutricionista
PUT	/api/pacientes/{pacienteId}/alertas/{alertaId}	`CriarAlertaRequest`	Nutricionista
DELETE	/api/pacientes/{pacienteId}/alertas/{alertaId}	RN23 — desativar é a única ação do módulo permitida ao paciente.	Autenticado
POST	/api/pacientes/{pacienteId}/alertas/{alertaId}/reativar		Nutricionista
POST	/api/pacientes/{pacienteId}/alertas/disparos	RN27 — o aplicativo informa cada disparo e seu desfecho. As notificações são locais, agendadas no próprio dispositivo, sem serviço externo. `RegistrarDisparoRequest`	Autenticado
GET	/api/pacientes/{pacienteId}/alertas/disparos		Autenticado
POST	/api/pacientes/{pacienteId}/alertas/verificar-expirados	RN35 — varredura de expiração, chamada pelo aplicativo ao iniciar sessão.	Autenticado

AntropometriaController

UC008 — Registrar Dados Antropométricos. Quem registra é o Nutricionista, e só ele. O paciente lê as próprias medidas e não as edita: peso e altura entram no cálculo da necessidade energética e, por consequência, no plano alimentar, e um número errado digitado pelo paciente contamina a prescrição inteira. Decisão do Mateus em 11/09/2026, que restringe o que o RF04.1 deixava em aberto. A leitura continua aberta aos dois perfis, decidida por paciente (RN33).

	ROTA	DESCRIÇÃO	PERFIL
POST	/api/pacientes/{pacienteId}/antropometria	`CriarRegistroAntropometricoRequest`	Autenticado
GET	/api/pacientes/{pacienteId}/antropometria	RF04.2 — histórico cronológico das medidas.	Autenticado
GET	/api/pacientes/{pacienteId}/antropometria/serie	RF08.2 — série de peso e IMC para o gráfico.	Autenticado
DELETE	/api/pacientes/{pacienteId}/antropometria/{registroId}		Autenticado

RegistroEmocionalController

UC012 — Registrar Emoção. O registro é do paciente; o diário e a análise de correlação ficam visíveis ao nutricionista responsável (RN04 do UC012).

	ROTA	DESCRIÇÃO	PERFIL
POST	/api/pacientes/{pacienteId}/emocoes	`CriarRegistroEmocionalRequest`	Autenticado
GET	/api/pacientes/{pacienteId}/emocoes		Autenticado

	ROTA	DESCRIÇÃO	PERFIL
GET	`/api/pacientes/{pacienteId}/emocoes/correlacao`	RF06.2 — análise da correlação entre emoções e variações glicêmicas. Exclusiva do painel do nutricionista.	Nutricionista
DELETE	`/api/pacientes/{pacienteId}/emocoes/{registroId}`		Autenticado

FavoritoController

RF09.3 — favoritos do paciente no repositório educativo. O acesso é o mesmo das demais rotas por paciente (RN33): o próprio paciente pelo app ou pela web, e o nutricionista responsável, que precisa enxergar o que o paciente guardou para orientar a conduta.

	ROTA	DESCRIÇÃO	PERFIL
GET	`/api/pacientes/{pacienteId}/favoritos`		Autenticado
PUT	`/api/pacientes/{pacienteId}/favoritos/conteudos/{conteudoId}`		Autenticado
DELETE	`/api/pacientes/{pacienteId}/favoritos/conteudos/{conteudoId}`		Autenticado
PUT	`/api/pacientes/{pacienteId}/favoritos/receitas/{receitaId}`		Autenticado
DELETE	`/api/pacientes/{pacienteId}/favoritos/receitas/{receitaId}`		Autenticado

GlicemiaController

UC004 — Registrar Glicemia e UC005 — Visualizar Histórico Glicêmico. O registro é ato do paciente pelo app; a consulta é dele e do nutricionista responsável (RN01 e RN02 do UC005), decidido por paciente pela RN33.

	ROTA	DESCRIÇÃO	PERFIL
POST	`/api/pacientes/{pacienteId}/glicemia`	`CriarRegistroGlicemiaRequest`	Autenticado
GET	`/api/pacientes/{pacienteId}/glicemia`	RN04 do UC005 — últimos 7 dias por padrão.	Autenticado
GET	`/api/pacientes/{pacienteId}/glicemia/serie`	RF08.1 — série do gráfico de evolução glicêmica.	Autenticado
DELETE	`/api/pacientes/{pacienteId}/glicemia/{registroId}`		Autenticado

NecessidadeEnergeticaController

UC006 — Calcular Necessidade Energética. Ator é o Nutricionista: a RN14 veda valores calóricos arbitrários, e a prescrição é ato do profissional.

	ROTA	DESCRIÇÃO	PERFIL
POST	`/api/pacientes/{pacienteId}/necessidade-energetica/simular`	UC006, passo 6 — calcula e devolve TMB, fator e VET sem persistir, para o nutricionista conferir antes de confirmar no passo 7. `CalcularVetRequest`	Nutricionista
POST	`/api/pacientes/{pacienteId}/necessidade-energetica`	UC006, passo 7 — confirma e grava o cálculo no prontuário. `CalcularVetRequest`	Nutricionista
GET	`/api/pacientes/{pacienteId}/necessidade-energetica`	RN04 do UC006 — histórico versionado, para rastreabilidade clínica.	Nutricionista

PlanoAlimentarController

UC007 — Elaborar Plano Alimentar. A elaboração é ato do Nutricionista; a leitura do plano vigente é liberada também ao paciente, que o consulta pelo aplicativo.

	ROTA	DESCRIÇÃO	PERFIL
GET	/api/pacientes/{pacienteId}/plano-alimentar	Plano vigente do paciente. Visível ao paciente e ao responsável (RN33).	Autenticado
GET	/api/pacientes/{pacienteId}/plano-alimentar/historico	RN17 — histórico completo de prescrições, incluindo as inativadas.	Autenticado
POST	/api/pacientes/{pacienteId}/plano-alimentar/distribuicao	UC007 A2 — prévia da distribuição de macronutrientes sobre o VET vigente, antes de o plano existir. Sem corpo, devolve a distribuição automática.	Nutricionista
POST	/api/pacientes/{pacienteId}/plano-alimentar	`CriarPlanoRequest`	Nutricionista
POST	/api/pacientes/{pacienteId}/plano-alimentar/{planoId}/duplicar	UC007 A3 — duplica um plano anterior como base do novo.	Nutricionista
POST	/api/pacientes/{pacienteId}/plano-alimentar/{planoId}/reativar	RN17 — retoma um plano do histórico, inativando o vigente.	Nutricionista
DELETE	/api/pacientes/{pacienteId}/plano-alimentar/{planoId}		Nutricionista

RelatorioController

Relatórios clínicos exportáveis em PDF — o RelatorioController do C4 e o RelatorioService do Diagrama de Classes V5.0.

	ROTA	DESCRIÇÃO	PERFIL
GET	/api/pacientes/{pacienteId}/relatorios/glicemico		Nutricionista
GET	/api/pacientes/{pacienteId}/relatorios/antropometrico		Nutricionista
GET	/api/pacientes/{pacienteId}/relatorios/completo		Nutricionista

ReceitaController

RF09.1 — Receitas. Cadastro pelo Nutricionista (RN28); despublicação exclusiva do Administrador (RN29).

	ROTA	DESCRIÇÃO	PERFIL
GET	/api/receitas	RF09.3 — o paciente busca por nome ou por ingrediente.	Autenticado
GET	/api/receitas/{id}		Autenticado
POST	/api/receitas	`CriarReceitaRequest`	Nutricionista
PUT	/api/receitas/{id}	`CriarReceitaRequest`	Nutricionista
DELETE	/api/receitas/{id}		Administrador
POST	/api/receitas/{id}/republicar		Administrador

ReferenciasController

Leitura das tabelas de referência que substituíram os enums (Diagrama de Classes V5.0), para preencher os selects dos formulários. O CRUD completo de cada uma entra quando houver demanda real de manutenção pelo Administrador.

	ROTA	DESCRIÇÃO	PERFIL
GET	/api/referencias/sexos-biologicos		Autenticado
GET	/api/referencias/tipos-diabetes		Autenticado

	ROTA	DESCRIÇÃO	PERFIL
GET	/api/referencias/contextos-glicemia		Autenticado
GET	/api/referencias/estados-emocionais		Autenticado
GET	/api/referencias/formulas-energeticas		Autenticado
GET	/api/referencias/niveis-atividade		Autenticado
GET	/api/referencias/tipos-alerta		Autenticado
GET	/api/referencias/tipos-conteudo		Autenticado
GET	/api/referencias/faixas-macronutrientes	Faixas da Diretriz da SBD para diabetes tipo 2, com o padrão que o sistema sugere. A tela usa para orientar o nutricionista sem impedir a conduta dele	Autenticado

9 Importação da tabela TACO

A tabela TACO é publicada em PDF e em planilha, e chega ao sistema como CSV. O arquivo real impõe um conjunto de particularidades que um leitor ingênuo não sobrevive.

PARTICULARIDADE	COMO É TRATADA
Codificação	Exports gerados pelo Excel saem em Windows-1252, não em UTF-8. Ler como UTF-8 corromperia os nomes acentuados — que são a chave de duplicidade da regra 11, e portanto o que impede o mesmo alimento de entrar duas vezes. O leitor tenta UTF-8 estrito e cai para Windows-1252 quando os bytes não formam UTF-8 válido.
Separador e decimal	CSV brasileiro usa ponto e vírgula como separador e vírgula como decimal, e nomes como "Arroz, integral, cozido" trazem vírgulas dentro do próprio campo. O separador é detectado pela primeira linha, e o milhar com ponto é tratado.
Marcadores no lugar de números	Traço indica quantidade desprezível porém presente, e vira zero. Asterisco e NA indicam valor não determinado, e viram ausência de dado. Zerar um valor não medido produziria um plano alimentar com contas erradas e nada na tela denunciando.
Títulos de seção	A planilha separa os alimentos por grupo, e o título ocupa a própria coluna de descrição. O que distingue um título de um alimento não é o valor ausente, e sim a célula vazia: a quarta edição publica asterisco em todos os macronutrientes de seis itens reais, entre eles o leite de vaca líquido e o sal. Decidir pelo valor nulo descartaria esses alimentos em silêncio.
Cabeçalhos variados	A tabela circula em versões com títulos diferentes. O mapeamento é por nome, com sinônimos aceitos para cada campo, e não por posição — arquivos com colunas a mais ou em outra ordem continuam funcionando.
Relatório de erros	Linhas inválidas não interrompem a importação: entram num relatório com o número da linha, o conteúdo e o motivo, exibido na tela ao final.

O que a fonte não fornece

LACUNA	ALCANCE
Índice glicêmico	A tabela não publica o dado. 31 dos 597 alimentos receberam o valor das International Tables of Glycemic Index and Glycemic Load Values 2021, só onde alimento e preparo correspondem com clareza; arroz, feijão, macarrão e mandioca ficaram sem correspondência segura. A auditoria item a item está em db/indice-glicemico.
Seis alimentos sem macronutrientes	Leite de vaca integral e desnatado líquidos, sal grosso, sal dietético, coco verde e iogurte de abacaxi trazem valor não determinado em energia, proteína, lipídeos e carboidrato.

10 Modelo de leitura do painel

O painel do nutricionista lê de uma projeção própria, separada das tabelas de escrita. Essa separação foi orientada na modelagem; a forma como ela é construída mudou depois da revisão do professor orientador.

ASPECTO	COMO FUNCIONA
Uma view, não uma tabela	O resumo clínico era uma tabela física que guardava indicadores já calculados. Guardar dado derivado de outras tabelas viola a terceira forma normal, e foi o que a orientação apontou. Hoje é uma view: cada coluna é calculada na hora da consulta, a partir das tabelas transacionais.
Sem atualização a cargo dos serviços	Antes, todo serviço que gravava dado clínico precisava lembrar de atualizar o resumo depois. Esquecer era deixar o painel mostrando número velho. Com a view não existe versão desatualizada para corrigir, e a camada de serviço perdeu cerca de cento e cinquenta linhas dedicadas a essa sincronização.
O que a view calcula	Última glicemia com contexto e data, média e percentual no alvo dos últimos sete dias, último índice de massa corporal com a classificação, situação do plano, contagem de disparos pendentes e dias sem registro. Medições e materiais removidos por exclusão lógica ficam de fora, como nas telas.
O relatório faz o contrário	O serviço de relatórios lê das tabelas de escrita, porque precisa do histórico completo e não de indicadores consolidados. A separação entre os dois está explícita no diagrama de componentes.

> **Consequência visível**
>
> O cartão de registros do dia conta pacientes que registraram hoje, e não a quantidade de registros. O protótipo rotula o cartão de outra forma, e a diferença está documentada.

11 Testes

São 138 testes em duas camadas. Os unitários rodam em cerca de dois segundos e não dependem de banco; os de integração sobem a aplicação inteira contra um banco descartável, criado e destruído a cada execução.

Cobertura

GRUPO	O QUE VERIFICA
Cálculos clínicos	Índice de massa corporal e classificação, relação cintura-quadril, Harris-Benedict e Mifflin-St Jeor conferidas termo a termo, valor energético, distribuição de macronutrientes, classificação glicêmica e cálculo de idade, incluindo a convenção para nascidos em 29 de fevereiro.
Leitura da tabela nutricional	Marcadores de traço e de valor não determinado, codificação Windows-1252, detecção de separador, reconhecimento de cabeçalhos em suas variações e a distinção entre título de seção e alimento sem valores.
Validações de entrada	Dígito verificador do CPF com sequências repetidas e entradas malformadas, política de senha, hash com salt por chamada e geração de senha provisória sem caracteres ambíguos.
Regras de negócio	Contra banco real: bloqueio progressivo nos três níveis, senha provisória, unicidade de e-mail contra conta desativada, pré-requisito do cálculo energético, soma dos macronutrientes, unicidade de plano ativo — incluindo a recusa do próprio banco a dois planos simultâneos — reclassificação do histórico glicêmico ao mudar a faixa alvo, e isolamento entre profissionais.

> **Verificação por mutação**
>
> Uma suíte que passa mas não falha quando o código quebra não protege nada. Foram introduzidos defeitos deliberados — divisão errada no índice de massa corporal, coeficiente alterado em Harris-Benedict, escalonamento do bloqueio achatado, reclassificação glicêmica removida, ordem de gravação do plano ativo invertida e marcador de traço tratado como ausência — e a suíte falhou em todos os casos.

12 Configuração e execução

Nenhum segredo fica no repositório. A aplicação lê do armazenamento de segredos do .NET, e o repositório traz apenas o arquivo de exemplo com os nomes das chaves.

Chaves de configuração

CHAVE	FUNÇÃO
`ConnectionStrings:Supabase`	Conexão com o banco. Usada por padrão.
`ConnectionStrings:Default`	Banco local, usado quando USAR_LOCAL for verdadeiro.
`USAR_LOCAL`	Alterna para o Postgres local, para trabalhar sem rede.
`Jwt:SigningKey`	Chave de assinatura do token. Mínimo de 32 bytes.
`Jwt:Issuer · Jwt:Audience`	Emissor e audiência validados no token.
`Jwt:ExpiresHours`	Validade do token. Padrão de 24 horas.
`Email:Habilitado`	Liga o envio por SMTP. Desligado, registra no log.
`Email:Host · Porta · Usuario · Senha`	Credenciais do servidor de e-mail.
`Email:UrlWeb`	Base usada para montar o link de redefinição de senha.
`Google:ClientId`	Identificador do cliente OAuth. Sem ele o login federado fica desligado.

Execução

```
./dev.sh up sobe API e front contra o Supabase ./dev.sh up --local sobe o Postgres em Docker e aponta a API para ele ./dev.sh test roda a suíte ./dev.sh logs api acompanha o log
```

Ambiente em container

O banco de desenvolvimento sobe por Docker Compose, em um serviço PostgreSQL com volume nomeado e verificação de saúde. Quem for trabalhar no projeto não precisa instalar banco na máquina: o `./dev.sh up --local` levanta o container, aplica as migrations pendentes e sobe a API apontada para ele. A porta publicada é 5433, escolhida para não colidir com um PostgreSQL já instalado no sistema.

A mesma containerização vale para a publicação: a API é distribuída como imagem, e é isso que determina quais hospedagens conseguem recebê-la.

Conexão com o Supabase

O host de conexão direta resolve apenas em IPv6 e nem sempre é alcançável. O acesso vai pelo pooler, que tem IPv4, e o usuário leva o identificador do projeto. O modo sessão, na porta 5432, é obrigatório para as migrations; o modo transação, na 6543, é mais eficiente em produção mas exige desabilitar prepared statements na string de conexão.

Primeiro acesso

O sistema não permite autocadastro: pacientes são criados por nutricionistas e nutricionistas por administradores. Em ambiente de desenvolvimento, um administrador inicial é criado automaticamente quando o banco está vazio, para a cadeia poder começar.

13 Defeitos corrigidos

Erros que compilavam sem aviso e só apareceram com o sistema em execução ou contra o banco real. Estão aqui porque cada um mostra um limite de confiar apenas na compilação.

DEFEITO	O QUE ACONTECIA
Validação ignorada em silêncio	Atributos de validação em parâmetros de construtor primário de record eram ignorados pelo ASP.NET, que lançava exceção ao validar o modelo. O login respondia erro interno. Os atributos precisam ficar no parâmetro, não na propriedade.
Claims remapeadas	O manipulador de JWT converte, por padrão, os nomes padronizados de identificador e e-mail para URIs de outro vocabulário. A leitura pelo nome original devolvia vazio, o que quebrou primeiro a rota de perfil e depois toda a recuperação de senha.
Ordem de gravação contra constraint	Ao trocar o plano ativo, o Entity Framework emitia a atualização que ativa antes da que desativa, e o índice único parcial recusava o lote. Se a regra estivesse apenas no serviço, o defeito teria passado e o paciente ficaria com dois planos ativos. A liberação do lugar agora é gravada antes da ocupação, dentro de uma transação.
Valor derivado congelado	Alterar a faixa glicêmica alvo não reclassificava os registros anteriores. O painel exibia a faixa nova com a contagem antiga — um número errado na frente de quem decide. A marcação de fora do alvo é valor derivado da faixa vigente, não um fato histórico.
Nomes de coluna divergentes do modelo	O conversor de nomes separava dígitos no lugar errado, produzindo nove colunas diferentes do documentado. Foi corrigido no conversor, e não coluna a coluna.
Alimento descartado como título de seção	A heurística que reconhece títulos de grupo na planilha decidia pelo valor nulo, e descartava seis alimentos reais que a tabela publica sem macronutrientes. Eles sumiam sem aparecer no relatório de erros.
Informação nutricional incompleta sem aviso	A marcação de completude das receitas checava apenas os macronutrientes, e dava por completa uma receita com óleo, cuja fibra a tabela não determina. O total de fibra saía subestimado em silêncio.
Administrador impedido de criar plano	A tabela de planos exige um nutricionista, e o administrador não é um — embora a regra 04 diga que ele herda as permissões. O plano criado por ele agora é atribuído ao profissional responsável pelo paciente.
Comparativo retroativo inconsistente	Um registro antropométrico lançado com data anterior comparava-se à medição mais recente, e não à imediatamente anterior no tempo. O mesmo registro mostrava uma variação no cadastro e outra no histórico.

14 Dívida técnica

O que ficou pendente, com o custo de cada item. Nada aqui impede o sistema de funcionar.

ITEM	DETALHE
Campos previstos nos casos de uso Exige colunas novas	CPF, data de nascimento e sexo do nutricionista; endereço do paciente; observações no registro antropométrico; nome do plano alimentar, hoje gravado no campo de objetivo.
Publicação com HTTPS Depende de decisão de hospedagem	A regra 31 exige comunicação criptografada. O redirecionamento e o cabeçalho de segurança já são aplicados fora do ambiente de desenvolvimento; falta definir onde publicar. Como a API roda em container, a hospedagem precisa aceitar imagem Docker — critério que descarta plataformas restritas a aplicações estáticas ou a funções em JavaScript.
Índice glicêmico parcial Limite da fonte	31 dos 597 alimentos têm o valor da tabela internacional de 2021. O restante não tem correspondência segura nela — não é carga pendente, é dado que a fonte não cobre. O nutricionista pode informar o valor, e a origem fica registrada.
Conta de e-mail sem senha de aplicativo Depende de credencial	A conta de envio existe e o serviço de SMTP está implementado. O Google recusa a senha comum da conta em conexões SMTP: é preciso gerar uma senha de aplicativo, com a verificação em duas etapas ativa.
Conexão em modo transação Ajuste na string de conexão	O acesso ao Supabase usa o modo sessão, exigido pelas migrations. Para produção, o modo transação é mais eficiente, mas precisa desabilitar prepared statements na string.
Cobertura de testes de serviço Ampliação da suíte	Os testes cobrem os cálculos clínicos e as regras críticas. Serviços como o do repositório educativo e o de relatórios não têm teste automatizado próprio.